

Relatório de Laboratório

Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º Período
Professor(a)	Danilo Maia
Laboratório	Lab02 - Assistentes de IA vs. Codificação Manual
Grupo (trio)	Athos Marques, Mateus Araujo, Rafael Moura
Link do repositório / GitHub Projects	https://github.com/users/RafaelMouraG/projects/7
Data de entrega	25/09/2026

1. Introdução

Ferramentas de IA generativa tornaram-se onipresentes no desenvolvimento de software, no entanto, ainda há pouca evidência controlada e reproduzível sobre seu real impacto em produtividade e qualidade (a maior parte do que se avalia baseia-se em evidência anedótica). Este laboratório investiga quantitativamente os efeitos do uso de um assistente de IA na resolução de tarefas de programação, através do acompanhamento de métricas de tempo, sucesso e manutenibilidade estrutural.

As Questões de Pesquisa (RQs) do estudo e suas respectivas hipóteses informais antes da coleta são:

- RQ1. O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?
- H0: O tempo mediano de resolução com IA é maior ou igual ao tempo manual ($T_{IA} \geq T_{Manual}$).
- H1: O tempo mediano de resolução com IA é menor que o tempo manual ($T_{IA} < T_{Manual}$).
- RQ2. O uso de assistente de IA reduz a quantidade de defeitos (testes que falham) no código produzido?
- H0: A taxa mediana de sucesso nos testes com IA é menor ou igual à da codificação manual ($S_{IA} \leq S_{Manual}$).
- H1: A taxa mediana de sucesso nos testes com IA é maior do que na codificação manual ($S_{IA} > S_{Manual}$).
- RQ3. O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?
- H0: Não há diferença estatisticamente significativa na complexidade ciclomática mediana e na duplicação de código entre as abordagens ($C_{IA} = C_{Manual}$).
- H1: Há diferença estatisticamente significativa na complexidade ciclomática mediana e/ou na duplicação de código entre as abordagens ($C_{IA} \neq C_{Manual}$).

Inovação proposta (30%): Além das métricas fundamentais exigidas (Tempo, Taxa de Sucesso, Complexidade e Duplicação), propomos a aferição do Índice de Manutenibilidade (MI - Maintainability Index) como métrica composta adicional para a RQ3.

2. Contexto

Este laboratório insere-se na S02 do projeto e consiste na execução controlada de Katas (exercícios de programação de dificuldade padronizada) em Python. A metodologia baseia-se num experimento crossover within-subject contrabalanceado (cada membro atua como controle de si mesmo, resolvendo parte dos katas manualmente e parte com auxílio da IA).

Como base teórica, as métricas deste estudo foram modeladas seguindo a abordagem GQM (Goal-Question-Metric) descrita por Basili, Caldiera & Rombach, estabelecendo os vínculos claros entre o objetivo macro (medir IA vs Humano) e a coleta instrumental via código. A ferramenta de IA eleita para todos os trials assistidos foi o Claude (da Anthropic), para uniformizar o contexto experimental de todo o trio.

3. Metodologia

3.1 Principais Desafios

Durante a fase de execução (Sprint 02), três grandes desafios e desvios (#60) afetaram a fluidez e precisão do laboratório:

- Sensibilidade do Parser de Duplicação: A ferramenta de duplicação (jscpd) não obteve volume de tokens ou linhas o suficiente para encontrar clones, dado que as respostas aos katas foram curtas (10 a 20 linhas).
- Tempo Viciado por Extração "One-Shot": Os tempos capturados na abordagem "com IA" foram irrealisticamente baixos (mediana ~41s), pois a interação resumiu-se ao desenvolvedor, pelo terminal (Claude Code), passar o enunciado inteiro do kata ao agente, que gerava a solução one-shot diretamente no arquivo, sem iteração incremental.
- Correção de Teste de Aceitação: Encontrou-se uma falha/ambiguidade estrutural no Kata 2 durante o processo de execução manual, que demandou correção extra, aumentando artificialmente a carga horária deste trial em específico.

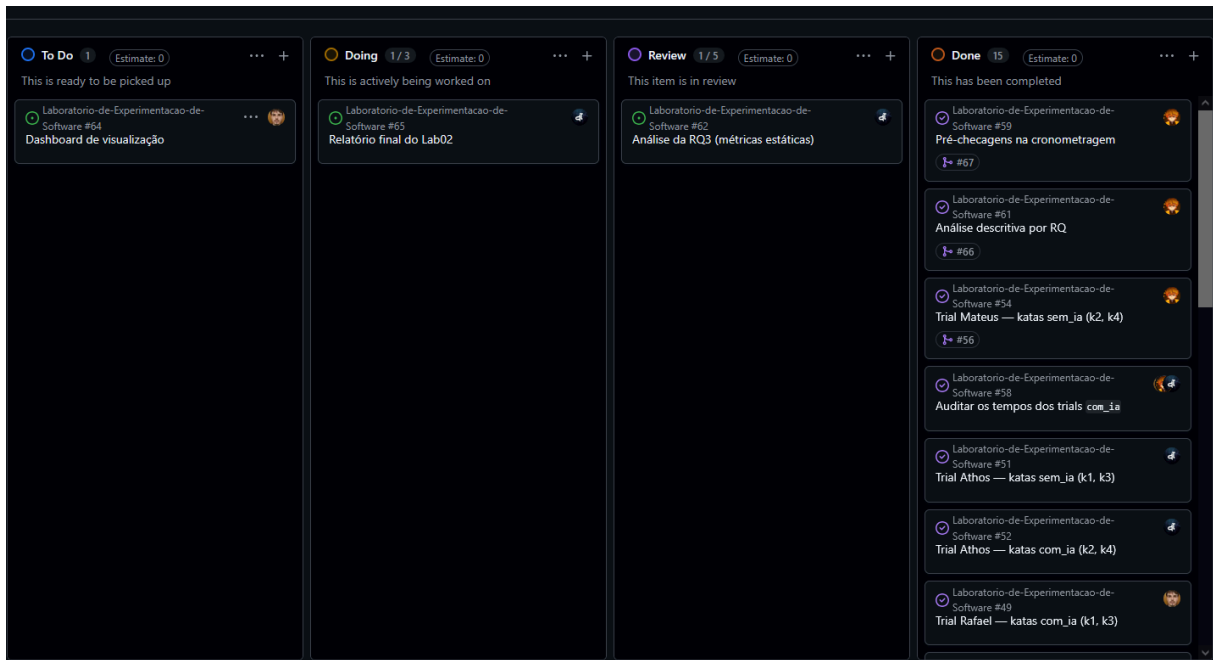
3.2 Tomadas de Decisão

- Uso da Mediana e IQR: Devido ao tamanho extremamente pequeno da amostra (12 trials no total) e ocorrência de outliers isolados, todas as consolidações adotaram a mediana e o IQR em vez de média e desvio padrão.
- Teste de Wilcoxon: Escolhemos o teste pareado de Wilcoxon, ideal para amostras não-paramétricas e experimentos within-subject.
- Censura no Time-box: Fixamos o time-box cravado em 35 minutos. Qualquer limite atingido não resultaria em exclusão (o que beneficiaria artificialmente os resultados com mais falhas), mas sim no registro censurado do trial com tempo 2100s.
- LOC como Variável de Controle Obrigatório: Adotamos o tamanho bruto (loc) como contra-peso nas métricas estáticas, pois os códigos de IA mostraram-se marginalmente mais prolixos, o que invalida uma leitura cega da complexidade ciclomática.

3.3 Etapas

A divisão de tarefas obedeceu ao pareamento no GitHub Projects. Uma visão das etapas de desenvolvimento:

Sprint	Entregas	Responsável(is)
Lab02S01	Desenho do experimento e scripts (cronometragem, métricas).	Athoosz, RafaelMouraG, mateusdsoc
Lab02S02	Execução do experimento (Katas alternados com e sem IA).	Athoosz, RafaelMouraG, mateusdsoc
Lab02S03	Análise estatística Wilcoxon e visualização gráfica.	Athoosz, RafaelMouraG, mateusdsoc
Relatório	Construção do relatório consolidado e evidências finais.	Athoosz, RafaelMouraG, mateusdsoc



Quadro Kanban (GitHub Projects) referente ao Sprint 02.

3.4 Ferramentas

- Linguagem / IDE: Python 3 (ambientes de edição configurados sem extensões de IA intrusivas nos cenários sem IA).
- Testes: pytest integrado ao script de cronometragem `src/cronometragem.py`.
- Métricas Estáticas: radon (complexidade cc, raw loc e maintainability index mi) e jscpd (duplicação).
- IA Generativa: Claude (Anthropic). (Interface: Claude Code, via terminal/CLI. Modelo: [Preencher modelo])
- Análise Estatística: Scripts em Python, utilizando validação do teste exato de Wilcoxon.
- Gestão: GitHub Projects (v2).

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ1	Time-to-green	Tempo percorrido até todos os testes passarem ou time-box ser atingido	Segundos	Script customizado (cronometragem.py)
RQ2	Taxa de sucesso	% de testes unitários passando ao final do limite de tempo	Porcentagem	Saída do pytest no script
RQ3	Complexidade média	Média da complexidade ciclomática de McCabe por função	Número	radon cc
RQ3	Duplicação de código	Percentual de código idêntico que burla o limiar mínimo de tokens	Porcentagem	jscpd
30%	Índice de Manutenibilidade	Fórmula de Maintainability Index do Halstead volume	Índice de 0 a 100	radon mi

3.6 Inovações Propostas pelo Grupo

Métrica adicional: Índice de Manutenibilidade (MI).

Propusemos a coleta e a avaliação do MI através do pacote radon mi. O racional baseia-se na insuficiência da métrica de complexidade isolada. Ao injetarmos o Índice de Manutenibilidade, unimos num só número o volume de Halstead, a complexidade ciclomática e o fator LOC. A hipótese central foi avaliar se a resposta one-shot do Claude criaria rotinas mais "complexas" estruturalmente, mas simultaneamente compensadas em um código mais manutenível (MI maior).

4. Resultados

4.1 Coleta de Dados

A amostra final computou exatos 12 trials. Não houve nenhum descarte de amostra nem dados ausentes.

Todos os trials foram finalizados antes do limite de tempo (0 censurados). Um outlier pontual na complexidade por linha (Athoosz/k3, valor 0.55) foi mantido, já que o método de Wilcoxon possui forte resiliência a picos isolados.

4.2 Visualização Gráfica

Os gráficos abaixo foram gerados pelo script `src/visualizacao.py` a partir de `data/trials.csv` (12 trials: 6 com IA, 6 sem IA).

RQ1. O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?

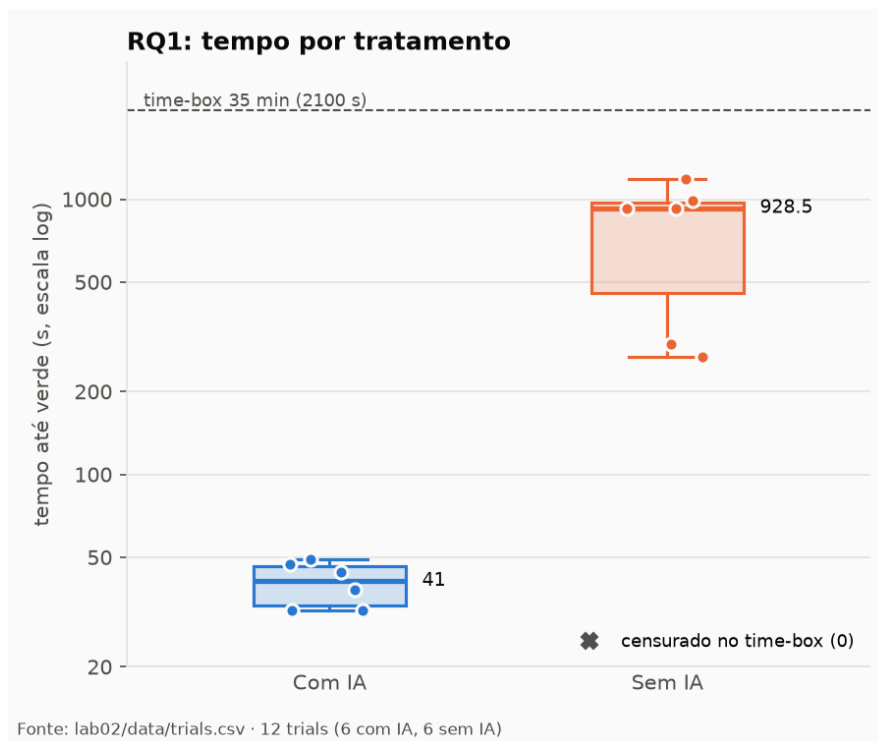


Figura 1 – Tempo até o verde por tratamento (escala log). Mediana de 41 s com IA contra 928,5 s sem IA; nenhum trial atingiu o time-box de 35 min.

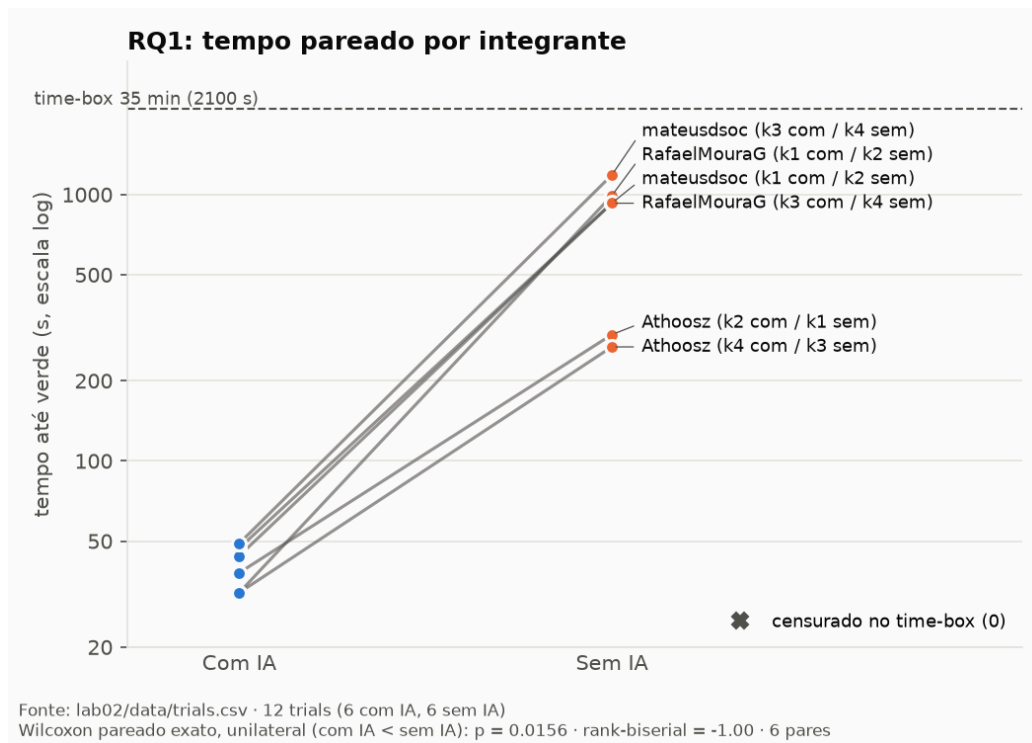


Figura 2 – Tempo pareado por integrante. Em todos os 6 pares o trial com IA foi mais rápido (Wilcoxon exato unilateral, $p = 0,0156$; rank-biserial = -1,00).

RQ2. O uso de assistente de IA reduz a quantidade de defeitos?

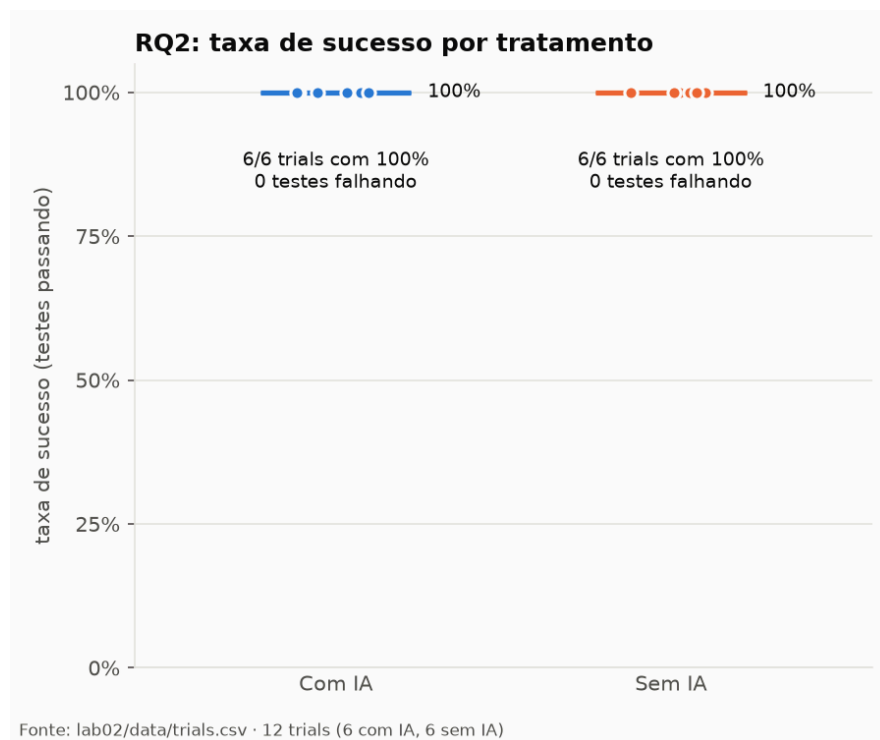


Figura 3 – Taxa de sucesso nos testes por tratamento. Todos os 12 trials terminaram com 100% dos testes passando, sem variação entre os grupos.

RQ3. O uso de assistente de IA altera a complexidade e a estrutura?

RQ3: métricas estáticas por tratamento

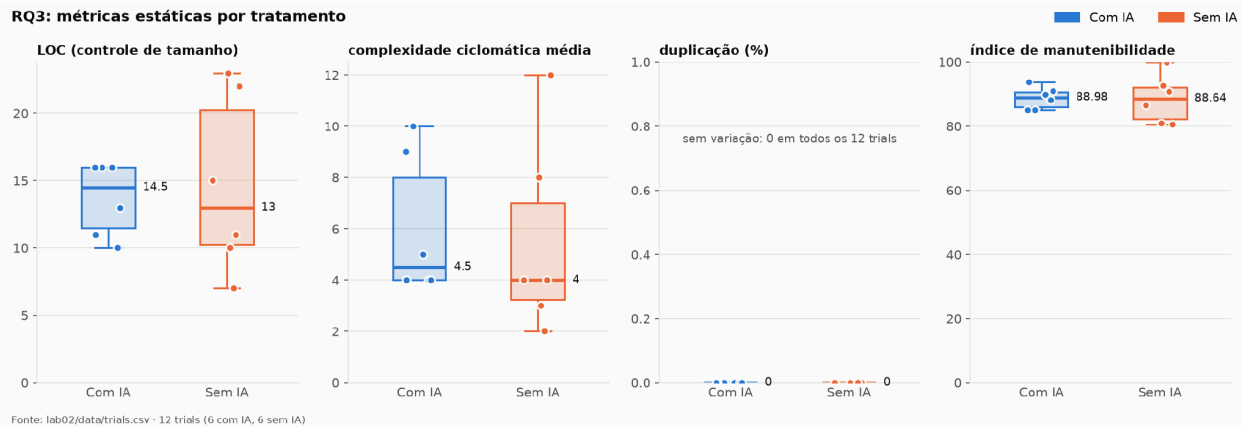


Figura 4 – Métricas estáticas por tratamento: LOC (mediana 14,5 com IA vs 13 sem IA), complexidade ciclomática média (4,5 vs 4), duplicação (0% em todos os trials) e Índice de Manutenibilidade (88,98 vs 88,64).

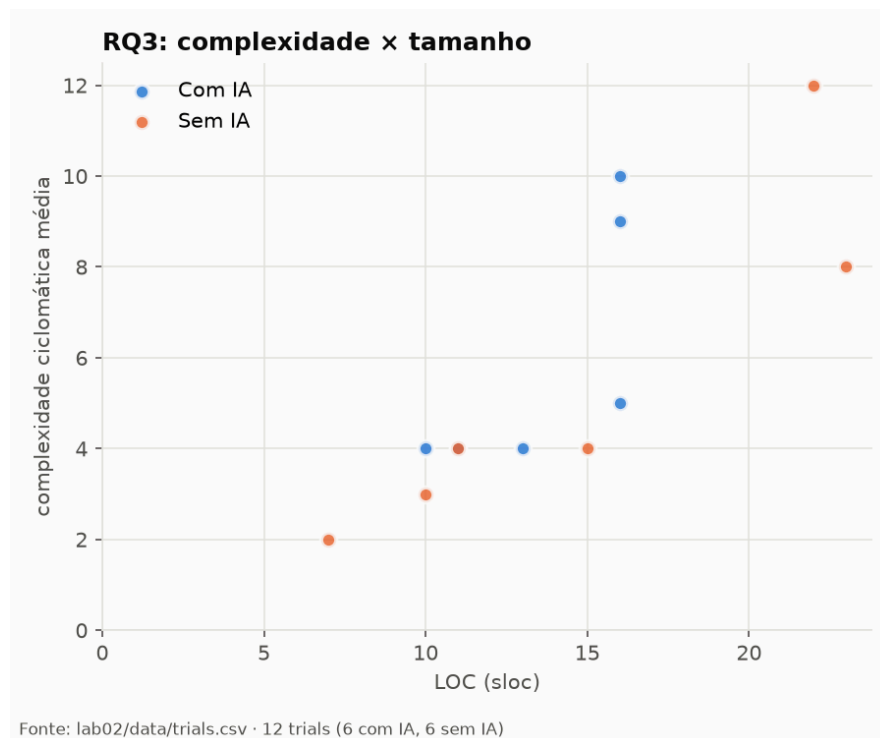


Figura 5 – Complexidade ciclomática x tamanho (LOC). A complexidade cresce com o tamanho do código nos dois tratamentos, o que justifica usar o LOC como variável de controle.

4.3 Discussão

- RQ1 (Tempo): A hipótese nula foi rejeitada ($p=0,0156$). Houve uma redução expressiva do tempo (de 928,5s manual para 41,0s com IA).
- RQ2 (Taxa de Sucesso): A hipótese nula não pôde ser rejeitada. Como ambos os métodos obtiveram 100% de sucesso, o teste de Wilcoxon empata de maneira perfeita e reflete que os katas, por serem curtos, não excederam a capacidade manual da equipe em gerar zero bugs.

- RQ3 e Inovação MI (Qualidade do Código): A hipótese nula não pôde ser rejeitada ($p=1,0$ para cc_media , $p=0,84$ para o mi de inovação). A duplicação cravou 0% estatisticamente para todo o repositório.

A métrica inovadora (Índice de Manutenibilidade) manteve a mediana estática (88,98 com IA vs 88,63 sem IA). Isso nos provou que, dentro de Katas algorítmicos compactos, a diferença arquitetônica ou de clareza inserida pela IA não superou a margem de erro ou a competência estrutural padrão dos próprios pesquisadores em regime manual.

Quanto às ameaças à validade, os desvios registrados (#60) foram vitais. O contrabalanceamento incompleto concentrou possíveis benefícios de aprendizagem (Athoosz iterou primeiro duas vezes com IA, depois sem IA, criando fadiga invertida). Além disso, o tempo aferido de 41s na IA configura uma falha de construto: mensuramos não o processo de "desenvolver em dupla com a máquina", e sim a "velocidade de delegar o kata inteiro ao agente no terminal e aguardar a geração do arquivo". A validade externa permanece restrita a projetos acadêmicos e katas one-shot.

5. Conclusão

O laboratório conseguiu evidenciar categoricamente a velocidade irrestrita com a qual uma IA generativa preenche respostas para algoritmos fechados (Katas). A queda de 15 minutos para ínfimos segundos consolida a capacidade da IA em tarefas padronizadas, ainda que essa medida seja uma proxy de interação bruta do modelo e não produtividade de arquitetura profunda.

Funcional e estruturalmente, as métricas de qualidade (testes, complexidade e o Índice de Manutenibilidade propício da nossa inovação) provaram-se inertes perante os cenários. Não se ganha manutenibilidade absurda, mas também não se infesta o código com complexidade espúria.

Como recomendações para o aprofundamento do estudo (futuro):

- Expandir o objeto experimental para Katas com múltiplos arquivos ou domínios de arquitetura (e não puramente algoritmos de um único arquivo Python).
- Restringir o uso do agente no terminal (Claude Code) a interações incrementais, ou compará-lo a um copiloto inline, medindo de forma transparente a velocidade de predição e pair-programming em vez da delegação one-shot do kata inteiro.

O piloto de benchmark apresentado na seção 6 já avança na primeira recomendação e trata a delegação autônoma como objeto de estudo, em vez de ameaça. Ele confirma o principal achado do Lab02: para modelos atuais, acertar a tarefa deixou de ser o diferencial, e a comparação precisa recair sobre processo, custo, desempenho e qualidade interna do código.

6. Estudo Complementar: Benchmark de Agentes de Programação (Piloto)

Em paralelo a este laboratório, foi conduzido, na mesma disciplina, o piloto de um benchmark de agentes de programação (llm-bench). Enquanto o Lab02 compara pessoas com e sem IA em katas, o benchmark compara agentes de IA entre si numa tarefa maior, executada sem intervenção humana.

6.1 Desenho do Estudo

Cada agente recebeu o mesmo enunciado: implementar do zero uma API HTTP de encurtar URLs (5 operações, persistência, concorrência e durabilidade), com stack livre.

Configuração	Modelo	Harness
Opus	claude-opus-5-5	Claude Code
Sol	gpt-6-sol	Codex
Astra	gpt-6-astra	Codex
Muse	muse-spark-1.3	OpenCode

Etapa	Como funciona
Execução	Container Docker isolado, rede restrita, sem busca na web e sem intervenção humana; uma tentativa por configuração
Aceitação	Avaliador automatizado verifica 35 requisitos obrigatórios que o agente nunca vê; a entrega é aceita só com 35/35
Processo	Tempo, chamadas de ferramenta, tokens e custo estimado
Desempenho	Latência e vazão da API sob carga (500 req/s e 32 conexões simultâneas)

6.2 Resultados

	Opus	Sol	Astra	Muse
Requisitos	35/35	35/35	35/35	35/35
Tempo	197 s	374 s	367 s	602 s
Chamadas de ferramenta	13	12	10	45
Custo estimado	US\$ 0,65	US\$ 0,28	US\$ 0,95	US\$ 0,63
Stack	Go	Python + SQLite	Python + SQLite	Python + SQLite
GET em carga normal (p50)	0,04 ms	1,32 ms	1,51 ms	1,50 ms
POST em carga normal (p50)	0,05 ms	42,3 ms	1,62 ms	1,57 ms

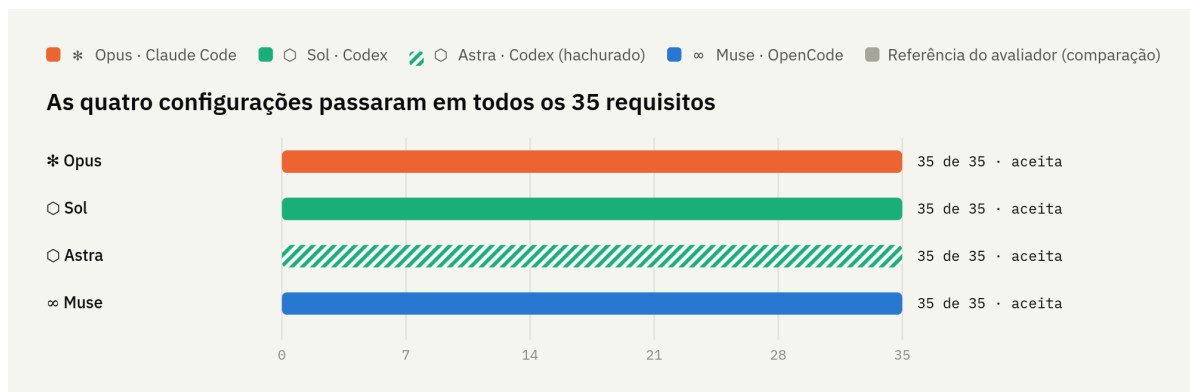


Figura 6 – Todas as configurações foram aceitas com 35 de 35 requisitos.

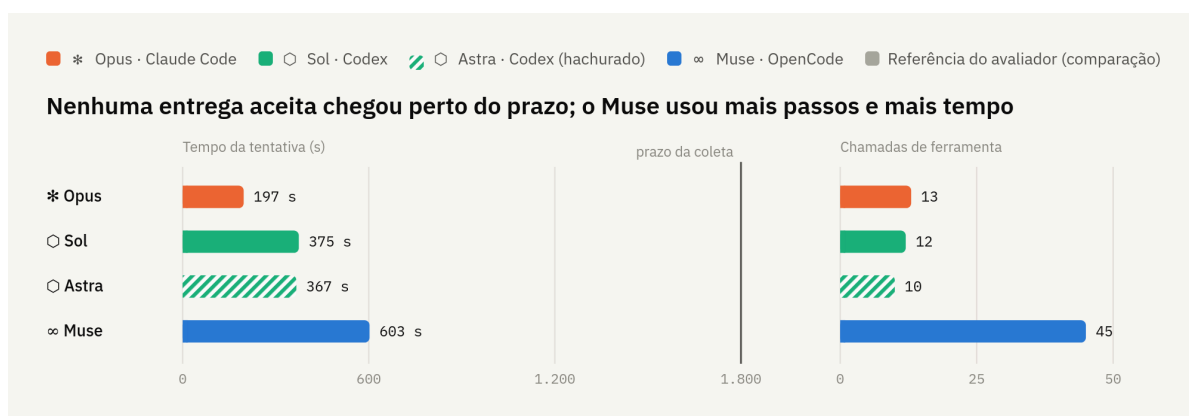


Figura 7 – Tempo de trabalho e chamadas de ferramenta. O Opus foi o mais rápido; o Muse, o que mais iterou.

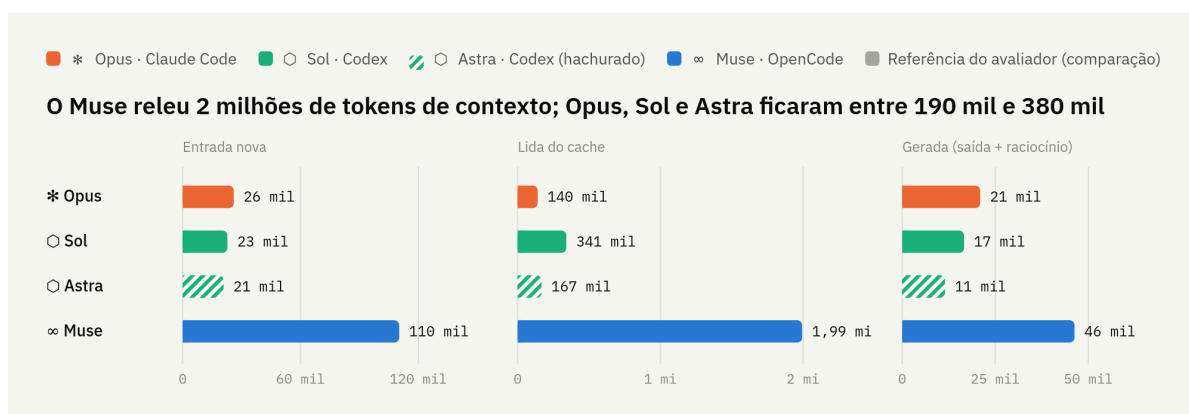


Figura 8 – Tokens por tentativa. O Muse releu cerca de 2 milhões de tokens de contexto.

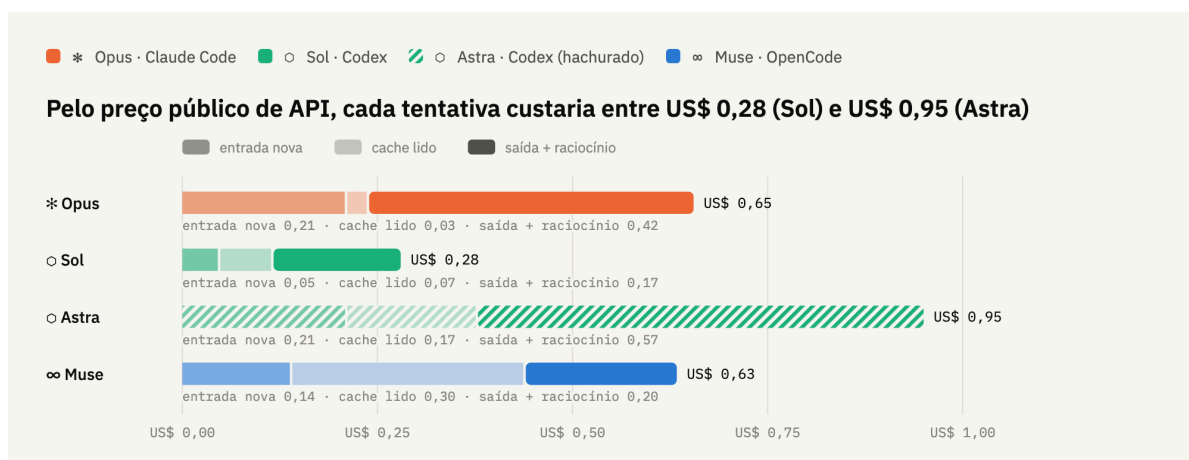


Figura 9 – Custo estimado pelo preço público de API, de US\$ 0,28 (Sol) a US\$ 0,95 (Astra), dividido por componente. A saída pesa mais em Opus, Sol e Astra; no Muse, o cache lido.

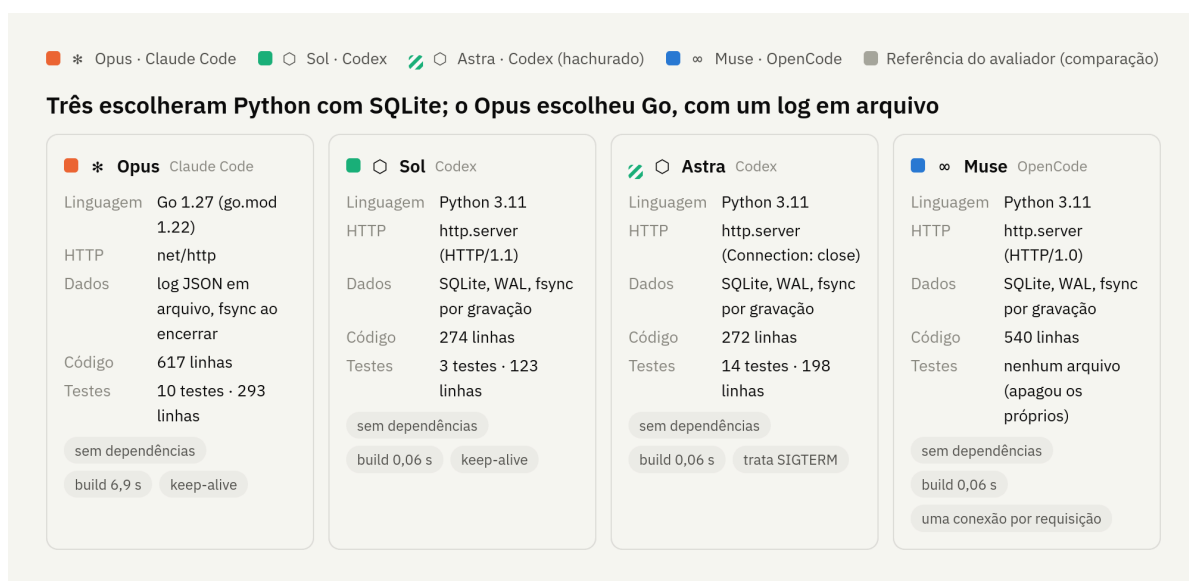


Figura 10 – Stack escolhida. Nenhuma entrega usou dependências externas.

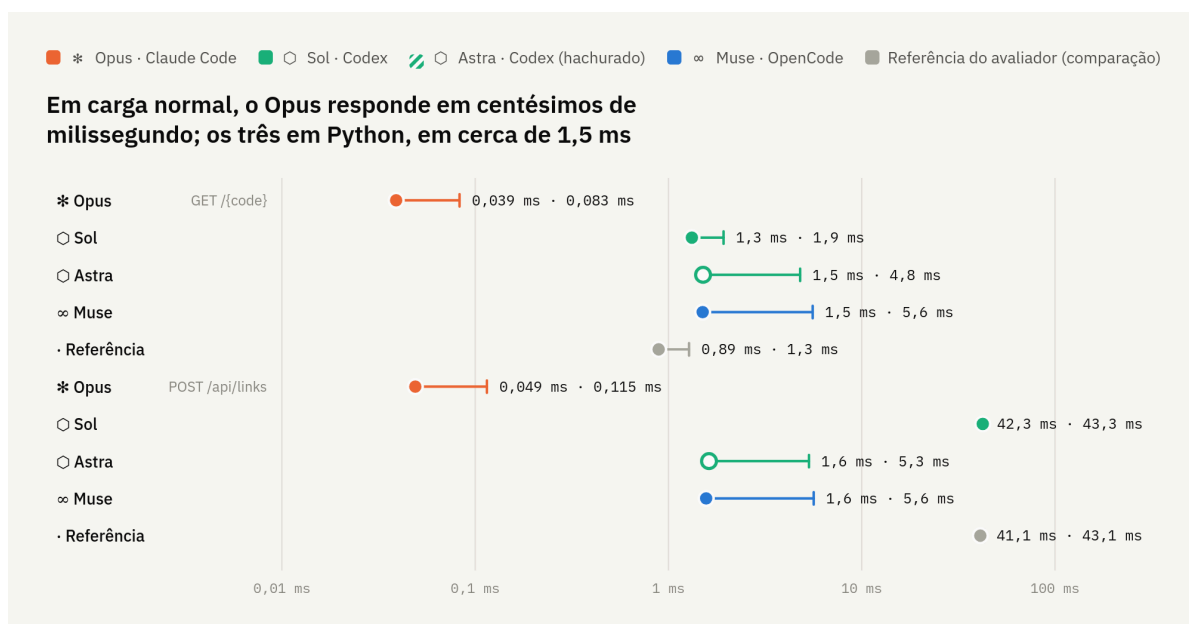


Figura 11 – Latência em carga normal (ponto = p50, ponta = p99; escala log).

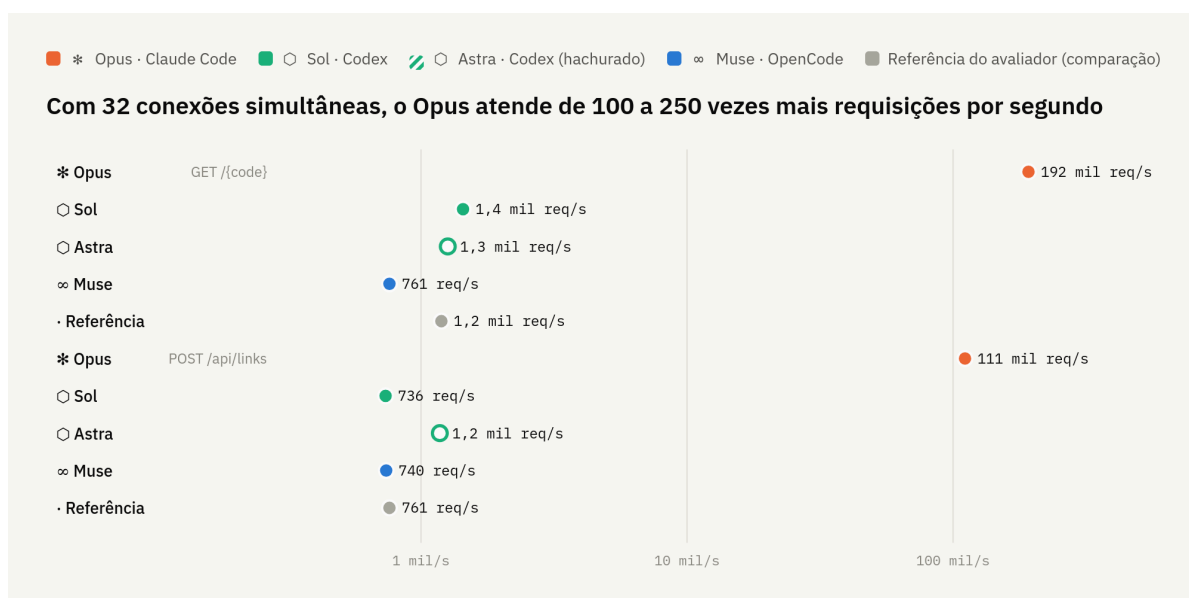


Figura 12 – Vazão com 32 conexões simultâneas (escala log).

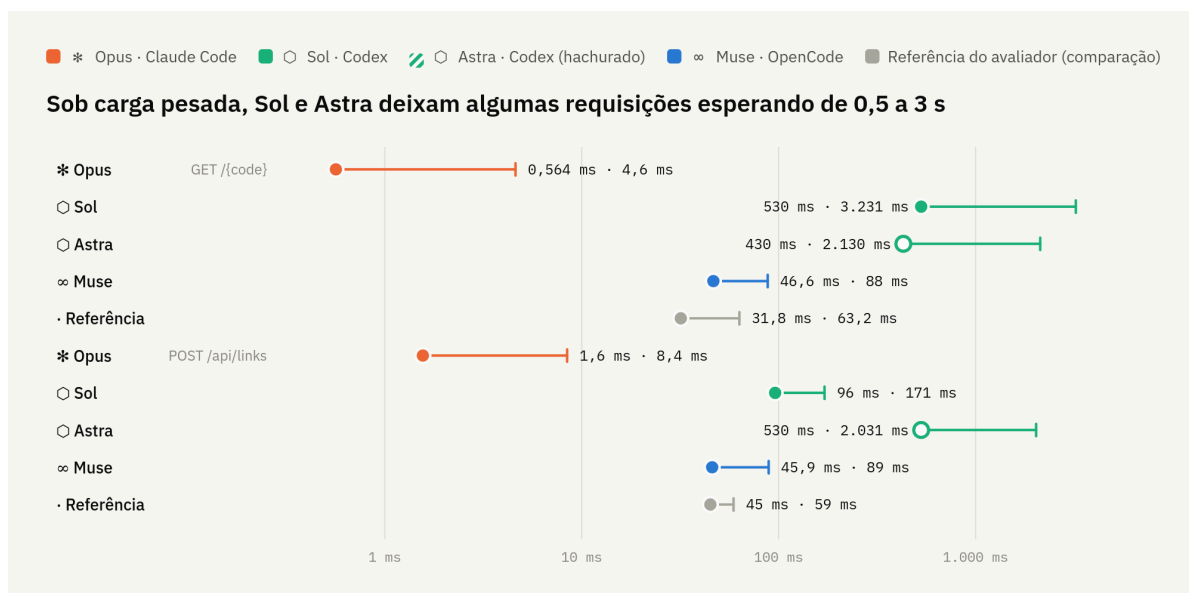


Figura 13 – Latência sob saturação (ponto = p99, ponta = pior caso; escala log).

Principais leituras:

- Aceitação não diferencia: todas passaram; a comparação fica no processo e no desempenho.
- A stack importa: a API em Go respondeu mais de 30 vezes mais rápido que as em Python.
- A implementação importa tanto quanto: com a mesma stack, um detalhe de HTTP deixou o POST do Sol em 42 ms, contra cerca de 1,6 ms de Astra e Muse.

6.3 Conexão com o Lab02

Aspecto	Lab02	Benchmark
O que compara	Pessoas com e sem IA	Agentes de IA entre si
Tarefa	Katas curtos, um arquivo	API completa, vários endpoints

Método	GQM + Wilcoxon	GQM + aceitação por contrato
Correção	100% dos testes em todos os trials	35/35 requisitos em todas as entregas
Onde apareceram diferenças	Tempo	Tempo, custo e desempenho
Qualidade interna	Sem diferença (katas pequenos demais)	Ainda não avaliada

Os dois estudos chegam ao mesmo achado: a correção saturou e não distinguiu os tratamentos. O benchmark também responde a duas limitações do Lab02: trata a delegação one-shot como objeto de estudo, e não como ameaça, e usa uma tarefa maior que um kata. Aplicar as métricas do Lab02 (radon, jscpd) às entregas do benchmark é um próximo passo natural.

6.4 Limitações

- Uma tentativa por configuração, sem medida de variação; resultados de piloto, não oficiais.
- Custo estimado por preço público de API, e latência medida sem rede real.

7. Referências

- BASILI, Victor; CALDIERA, Gianluigi; ROMBACH, H. Dieter. The goal question metric approach. 1994.
- LAB02. Enunciado e documentação oficial da disciplina. 2026.

